# Linux Interrupt Handling Framework

***A Practical Guide to ISR, Shared IRQ, Top Half & Bottom Half using Workqueues***

## Introduction

An **interrupt** is a hardware or software signal that temporarily halts the CPU's current execution and transfers control to an **Interrupt Service Routine (ISR)** to handle an event immediately.

## Need of Interrupts

- Enable immediate response to external events.
- Eliminate continuous polling of devices.
- Improve CPU utilization and system responsiveness.
- Support real-time event handling. **Polling vs Interrupts**

## Polling vs Interrupts

| Polling | Interrupts |
|---|---|
| CPU continuously checks device status | Device notifies CPU only when an event occurs |
| High CPU utilization | Efficient CPU utilization |
| Higher latency | Faster response time |
| Wastes processing cycles | Event-driven execution |

## Real-World Applications

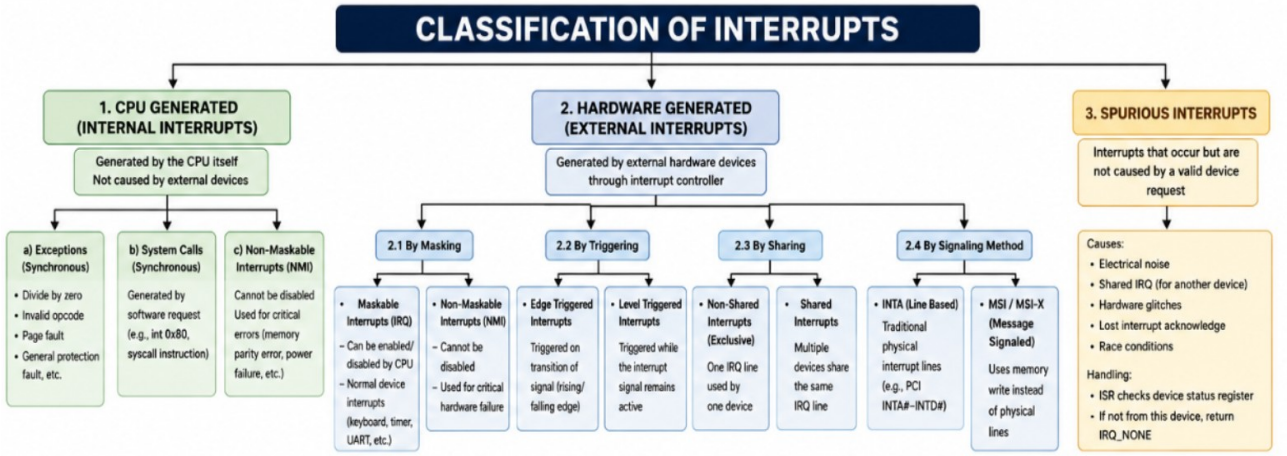
- USB Device Drivers
- Network Interface Cards (NIC)
- Keyboard & Mouse Drivers
- SPI / I²C Sensor Drivers
- Storage Controllers (NVMe, SATA)
- Automotive ECUs
- Industrial Automation Systems
- Embedded Linux & IoT Devices

## Advantages of Interrupts

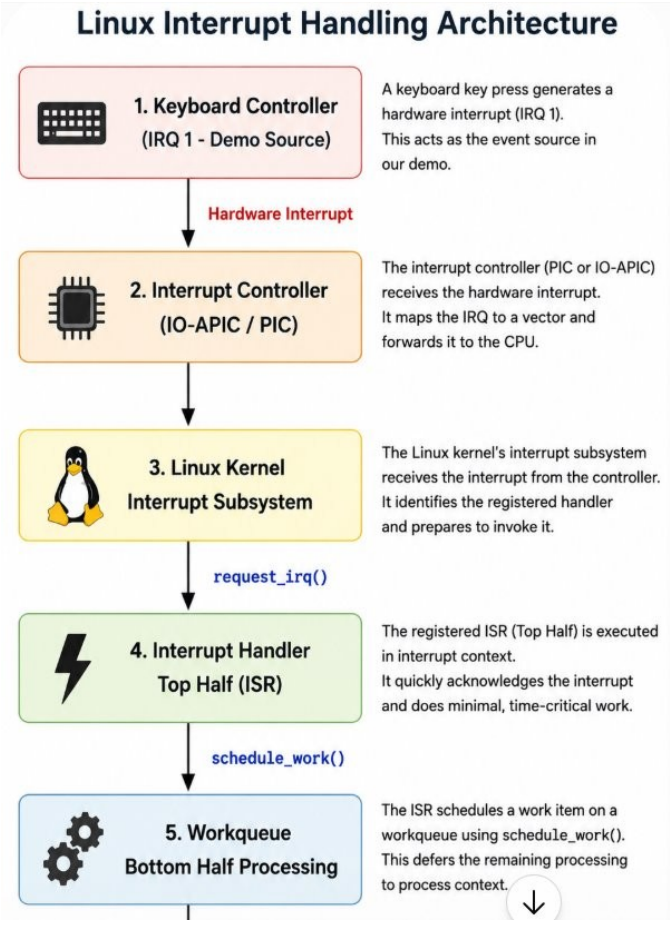
- Event-driven processing
- Reduced CPU overhead
- Faster response to hardware events
- Improved system performance
- Better resource utilization
- Foundation for real-time systems

# Classification of Interrupts

Interrupts are classified based on their source and purpose, enabling the operating system to respond efficiently to different events. Understanding these interrupt types helps developers design reliable, responsive, and high-performance Linux device drivers for real-time and embedded systems.




# Linux Interrupt Handling Architecture



# Interrupt processing mechanisms(Interrupt Handling)

Interrupt handling is the Linux kernel mechanism for responding to hardware or software interrupt events efficiently. It divides interrupt processing into **Top Half (immediate processing)** and **Bottom Half (deferred processing)** to minimize interrupt latency and improve overall system performance.

# 1.Interrupt Service Routine (ISR) – Top Half

The **Interrupt Service Routine (ISR)**, also called the **Top Half**, is the first function executed when a hardware interrupt occurs. It runs in **interrupt context**, performs only time-critical tasks such as acknowledging the interrupt, and schedules deferred processing. Since it executes in interrupt context, it **must be fast and cannot sleep or block**.

**Key API:** `request_irq()`

# 2.Bottom Half

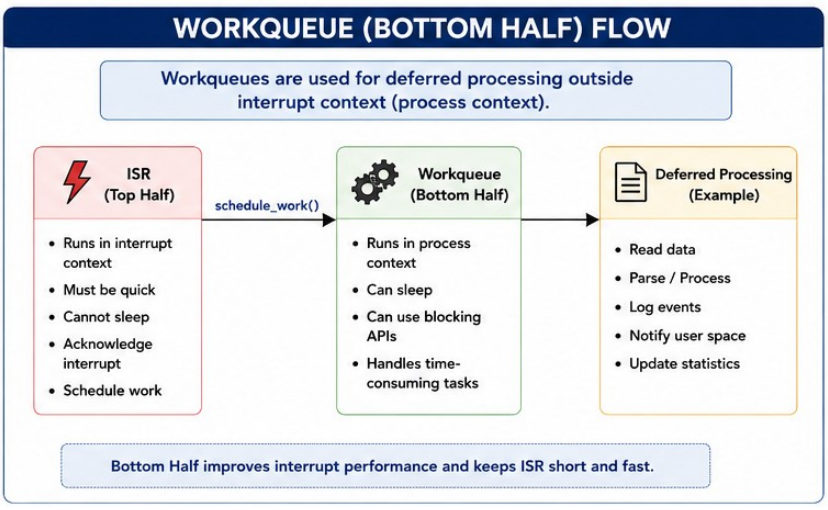
The **Bottom Half** performs the non-critical or time-consuming tasks that are deferred by the ISR. It executes **after the ISR** and allows lengthy operations to be completed without delaying the handling of other interrupts. Bottom Half mechanisms improve overall system responsiveness and interrupt latency. The **Bottom Half** can be implemented using techniques such as **SoftIRQ, Tasklets, Workqueues, and Threaded IRQs**.

Here in this project I used the Workqueue Bottom half mechanism.

# Workqueue

A **Workqueue** is a Linux Bottom Half mechanism that executes deferred tasks in **process context**. Unlike the ISR, workqueue functions **can sleep, allocate memory, and perform blocking operations**, making them suitable for complex processing such as device communication, logging, or data handling.

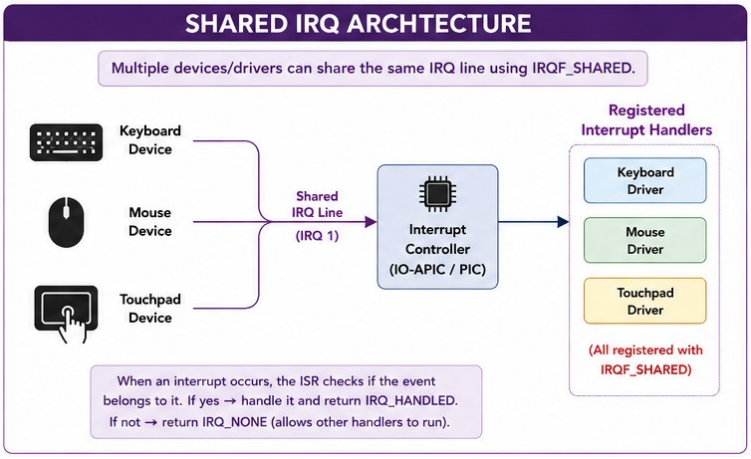
**Key APIs:** `INIT_WORK()`, `schedule_work()`



# Shared IRQ

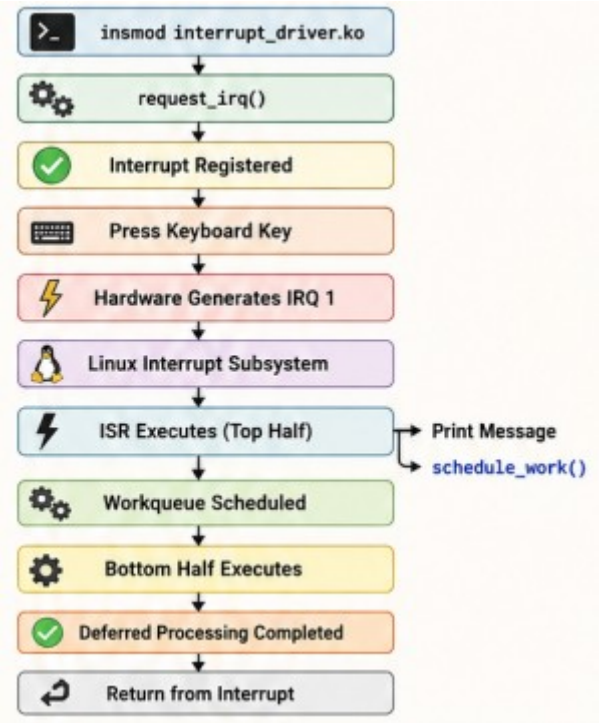
A **Shared IRQ** allows multiple devices or drivers to use the **same interrupt line**, reducing hardware resource usage. Drivers register a shared interrupt using the `IRQF_SHARED` flag and must verify whether the interrupt originated from their own device before handling it. If not, they should return `IRQ_NONE`.

**Key Flag:** `IRQF_SHARED`




# Linux Interrupt Execution Workflow

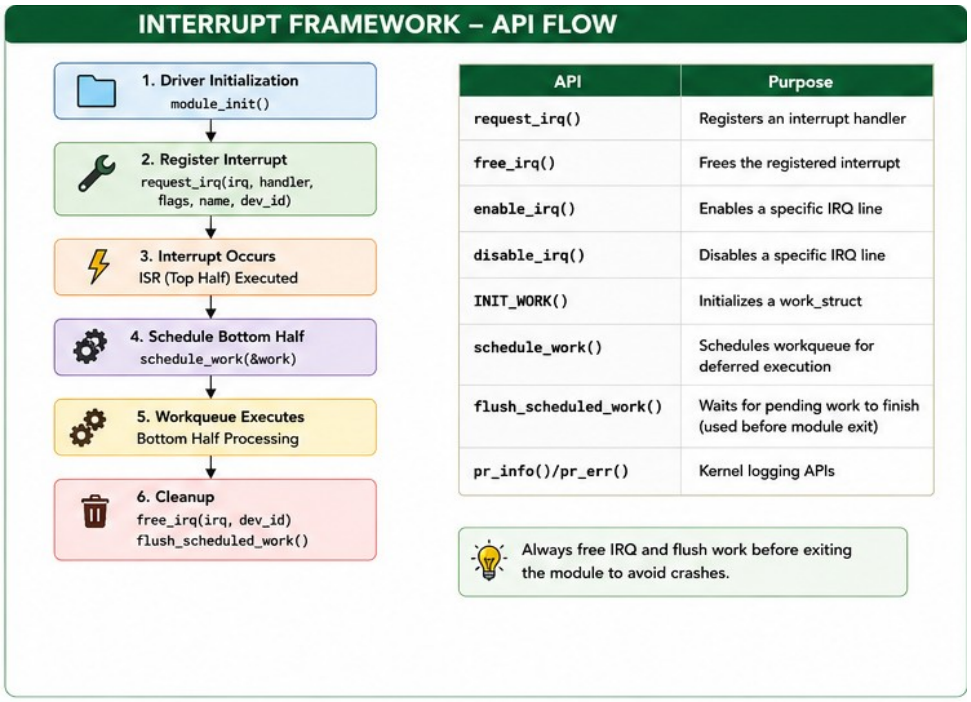
The following figure illustrates the complete **Linux interrupt handling workflow**. After the driver is loaded and the interrupt is registered using `request_irq()`, a keyboard event generates **IRQ 1**, invoking the **Interrupt Service Routine (Top Half)**. The ISR performs minimal processing, prints a message, and schedules deferred work using `schedule_work()`. The **Workqueue (Bottom Half)** then completes the remaining processing before returning control to the interrupted task.



# Important Interrupt APIs

The Linux kernel provides several APIs to register, manage, and release interrupts.

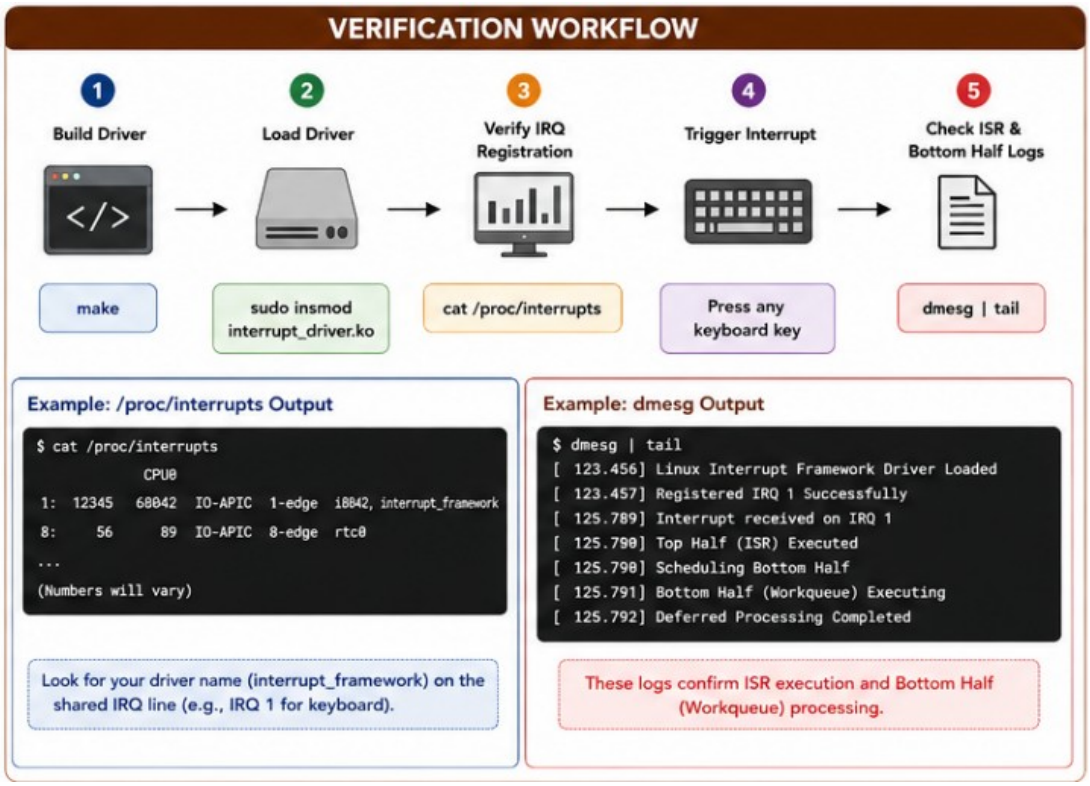
| API | Purpose |
|---|---|
| `request_irq()` | Registers an interrupt handler (ISR) |
| `free_irq()` | Releases the registered interrupt |
| `INIT_WORK()` | Initializes a workqueue item |
| `schedule_work()` | Schedules Bottom Half execution |
| `flush_scheduled_work()` | Waits for pending work before module removal |
| `enable_irq()` | Enables a disabled interrupt |
| `disable_irq()` | Disables an interrupt line |




# Verification Flow

The interrupt handling framework is verified by **building and loading the kernel module**, confirming interrupt registration through **`/proc/interrupts`**, generating a keyboard interrupt,

and validating **ISR** and **Bottom Half** execution using **`dmesg`**. Finally, the module is unloaded to ensure proper interrupt deregistration and resource cleanup.




# Implementation Results

# 1.Project Directory Structure

```
engineer@Deepa:~/Documents/personal/project-Intr_framework/code$ ls
interrupt_driver.c  Makefile
```

**Screenshot:** Project folder showing `interrupt_driver.c`, `Makefile`

# 2. Kernel Module Compilation

**Command: $make**

```
engineer@Deepa:~/Documents/personal/project-Intr_framework/code$ make
make -C /lib/modules/6.17.0-35-generic/build M=/home/engineer/Documents/personal/project-Intr_framework/code modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-35-generic'
make[2]: Entering directory '/home/engineer/Documents/personal/project-Intr_framework/code'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0
  You are using:           gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  CC [M]  interrupt_driver.o
  MODPOST Module.symvers
  CC [M]  interrupt_driver.mod.o
  CC [M]  .module-common.o
  LD [M]  interrupt_driver.ko
  BTF [M] interrupt_driver.ko
Skipping BTF generation for interrupt_driver.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/engineer/Documents/personal/project-Intr_framework/code'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-35-generic'
```

**Screenshot:** Successful compilation of `interrupt_driver.ko`.

# 3.Generated Build Artifacts

**Command: $ls -l**

```
engineer@Deepa:~/Documents/personal/project-Intr_framework/code$ ls -l
total 672
-rw-rw-r-- 1 engineer engineer   4514 Jun 30 14:07 interrupt_driver.c
-rw-rw-r-- 1 engineer engineer 333728 Jun 30 14:09 interrupt_driver.ko
-rw-rw-r-- 1 engineer engineer     21 Jun 30 14:09 interrupt_driver.mod
-rw-rw-r-- 1 engineer engineer   1468 Jun 30 14:09 interrupt_driver.mod.c
-rw-rw-r-- 1 engineer engineer 152784 Jun 30 14:09 interrupt_driver.mod.o
-rw-rw-r-- 1 engineer engineer 171992 Jun 30 14:09 interrupt_driver.o
-rw-rw-r-- 1 engineer engineer   1694 Jun 30 14:20 Makefile
-rw-rw-r-- 1 engineer engineer     19 Jun 30 14:09 modules.order
-rw-rw-r-- 1 engineer engineer      0 Jun 30 14:09 Module.symvers
```

**Screenshot:** Generated files and source file

| File | Purpose |
|---|---|
| **interrupt_driver.c** | Source code of the Linux kernel interrupt driver implementation. |
| **interrupt_driver.o** | Object file generated by compiling `interrupt_driver.c`. |
| **interrupt_driver.mod.c** | Auto-generated source file containing kernel module metadata (module information, versioning, init/exit references). |
| **interrupt_driver.mod.o** | Compiled object file of `interrupt_driver.mod.c`. |
| **interrupt_driver.mod** | Auto-generated file used by the kernel build system to track module information. |
| **interrupt_driver.ko** | Final loadable **Kernel Object (Kernel Module)** that can be loaded using `insmod`. |
| **Makefile** | Build script used by the Linux kernel build system to compile the module. |
| **modules.order** | Lists the order in which kernel modules are built and installed. |
| **Module.symvers** | Contains exported kernel symbols and version information used when building modules that depend on other kernel modules. |

# 4. Loading the Kernel Module

**Command:** `$sudo insmod interrupt_driver.ko`

```
engineer@Deepa:~/Documents/personal/project-Intr_framework/code$ sudo insmod interrupt_driver.ko
```

**Screenshot:** Terminal showing successful module loading.

# 5. Verifying Loaded Module

**Command:** `$lsmod | grep interrupt_driver`

```
engineer@Deepa:~/Documents/personal/project-Intr_framework/code$ lsmod | grep interrupt_driver
interrupt_driver       12288  0
engineer@Deepa:~/Documents/personal/project-Intr_framework/code$ modinfo interrupt_driver.ko
filename:       /home/engineer/Documents/personal/project-Intr_framework/code/interrupt_driver.ko
version:        2.0
description:    Linux Interrupt Handling Framework using ISR, Shared IRQ and Workqueue
author:         Deepa
license:        GPL
srcversion:     3C226BB1115053946545F43
depends:
name:           interrupt_driver
retpoline:      Y
vermagic:       6.17.0-35-generic SMP preempt mod_unload modversions
parm:           irq:IRQ Number (int)
```

**Screenshot:** Module listed in the kernel.

Module information is verified using **$modinfo  interrupt_driver.ko**

# 6.Kernel Log After Module Loading

```
engineer@Deepa: $ sudo dmesg -w
[sudo] password for engineer:
[102314.030406] Deferred Processing Completed
[102314.030406] ========================================
[102314.109133] Interrupt received on IRQ 1

[102314.109141] ********** TOP HALF ********************
[102314.109141] Interrupt Received
[102314.109142] IRQ Number       : 1
[102314.109142] CPU              : 9
[102314.109143] Interrupt Count  : 52
[102314.109144] Bottom Half Scheduled
[102314.109144] ****************************************

[102314.109164] ============== BOTTOM HALF ================
[102314.109164] Running in Process Context
[102314.109165] Current Process : kworker/9:0
[102314.109166] Deferred Processing Completed
[102314.109166] ========================================
[102314.232717] Interrupt received on IRQ 1

[102314.232725] ********** TOP HALF ********************
[102314.232725] Interrupt Received
[102314.232726] IRQ Number       : 1
[102314.232726] CPU              : 9
[102314.232727] Interrupt Count  : 53
[102314.232728] Bottom Half Scheduled
[102314.232728] ****************************************

[102314.232748] ============== BOTTOM HALF ================
[102314.232749] Running in Process Context
[102314.232749] Current Process : kworker/9:0
[102314.232750] Deferred Processing Completed
[102314.232750] ========================================
[102314.314850] Interrupt received on IRQ 1

[102314.314857] ********** TOP HALF ********************
[102314.314858] Interrupt Received
[102314.314858] IRQ Number       : 1
[102314.314859] CPU              : 9
[102314.314859] Interrupt Count  : 54
[102314.314860] Bottom Half Scheduled
[102314.314861] ****************************************
```

**Command:** `$dmesg`

**Screenshot:** Driver initialization messages and successful IRQ registration and execution.

ISR execution log displaying:

- Interrupt received
- IRQ number
- CPU ID
- Interrupt count

Workqueue execution showing deferred processing.

Interrupt Counter Verification

**Action:** Press multiple keyboard keys.

Observed the Interrupt count increases.

# 7. Verifying Interrupt Registration, Shared IRQ

**Command:** `$cat /proc/interrupts`

```
engineer@Deepa:~/Documents/b8/drtver$ cat /proc/interrupts
           CPU0       CPU1       CPU2       CPU3       CPU4       CPU5       CPU6       CPU7       CPU8       CPU9       CPU10      CPU11      CPU12      CPU13
  1:          0          0          0          0          0          0          0          0          0      22280          0          0          0          0 IR-IO-APIC    1-edge      i8042, interrupt_framework
```

**Screenshot:** IRQ 1 showing both `i8042` and `interrupt_framework`.

| Field | Meaning |
|---|---|
| `1` | IRQ Number |
| `IR-IO-APIC` | Interrupt controller |
| `1-edge` | Edge-triggered interrupt |
| `i8042` | Keyboard controller driver |
| `interrupt_framework` | Your kernel module |

Because **both `i8042` and `interrupt_framework` appear on the same IRQ (IRQ 1)**, this confirms that **IRQ 1 is being shared**, which is exactly what `IRQF_SHARED` enables.

# 8. Module Removal

**Command:** `$sudo rmmod interrupt_driver`

```
engineer@Deepa:~/Documents/personal/project-Intr_framework/code$ sudo rmmod interrupt_driver
engineer@Deepa:~/Documents/personal/project-Intr_framework/code$ lsmod | grep interrupt_driver
```

**Screenshot:** Driver unloaded successfully. Later verified using $lsmod

# 9. Kernel Log After Module Removal

**Command:** `$dmesg | tail -20`

```
[104925.832181] ================================================
[104925.832182] Interrupt Framework Driver Unloaded
[104925.832183] Total Interrupts Handled : 161
[104925.832183] IRQ Released Successfully
[104925.832184] ================================================
```

**Screenshot:** Interrupt released and cleanup completed.

# 10. Functional Verification Summary

| Test Case | Result |
|---|---|
| Module Compilation | Pass |
| Module Loading | Pass |
| IRQ Registration | Pass |
| ISR Execution | Pass |
| Bottom Half Scheduling | Pass |
| Workqueue Execution | Pass |
| Shared IRQ Support | Pass |
| Interrupt Counter | Pass |
| Kernel Logging | Pass |
| Module Unloading | Pass |
| Resource Cleanup | Pass |

# 11. Performance Observation

- ISR executes immediately upon interrupt reception.
- Time-consuming processing is deferred to the Workqueue.
- Shared IRQ registration functions correctly.
- Resources are released safely during module unload.

### Learning Outcomes

- Successfully implemented interrupt registration using `request_irq()`.
- Demonstrated **Top Half (ISR)** and **Bottom Half (Workqueue)** processing.
- Verified shared interrupt handling with `IRQF_SHARED`.
- Validated interrupt handling using `/proc/interrupts` and `dmesg`.
- Performed proper cleanup using `free_irq()` and `flush_work()`.

This implementation provides a practical understanding of Linux interrupt handling and serves as a foundation for developing production-grade Linux device drivers.

Author

Deepa